

1 Core Utilities

1.1 Type conversion (ALWAYS use these)

```
# uint8 [0,255] -> float32 [0,1]
r = img_u8.astype(np.float32) / 255.0

# float [0,1] -> uint8 [0,255] (clip first!)
out = (np.clip(f, 0, 1) * 255 + 0.5).astype(np.uint8)

# any float -> uint8 (min-max normalise)
def normalise(arr):
    mn, mx = arr.min(), arr.max()
    if mx == mn: return np.zeros_like(arr, np.uint8)
    return ((arr-mn)/(mx-mn)*255).astype(np.uint8)
```

Safe workflow: uint8 $\rightarrow$ float $\rightarrow$ compute $\rightarrow$ clip $\rightarrow$ uint8.
Direct uint8 arithmetic with NumPy **wraps around** (overflow/underflow)!

1.2 Read / Show / Save

```
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
plt.imshow(img, cmap='gray', vmin=0, vmax=255)
cv2.imwrite(path, img)
```

2 Part 1A Gamma Correction

Formula: $s = r^\gamma$, $r \in [0, 1]$

γ	Effect
< 1	Brighter (raise shadows)
$= 1$	Identity
> 1	Darker (compress shadows)

```
# NumPy
s = np.power(to_float01(img), gamma)
result = to_uint8(s)

# OpenCV (LUT = precomputed table, faster)
table = np.array([(i/255)**gamma)*255
                  for i in range(256)], dtype=np.uint8)
result = cv2.LUT(img_u8, table)
```

When to use: Preprocess before Otsu on dark images ($\gamma < 1$). Can make histogram more bimodal $\Rightarrow$ better Otsu threshold.

3 Part 1B Arithmetic Operations

$$S_+ = \alpha A + \beta B, \quad S_- = \alpha A - \beta B, \quad S_\times = A \odot B, \quad S_{\div} = \frac{A}{B+\epsilon}$$

```
# NumPy (float + clip) flexible
A, B = to_float01(a), to_float01(b)
add_ = to_uint8(A + B)      # clip at 255
sub_ = to_uint8(A - B)      # clip at 0
mul_ = to_uint8(A * B)      # always <= 1
div_ = to_uint8(A / (B + 1e-6)) # can exceed 1!
```

```
# OpenCV (saturating uint8) fast
cv2.add(a, b)      # saturates at 255
cv2.subtract(a, b) # saturates at 0
cv2.multiply(a, b, scale=...)
cv2.divide(a, b, scale=...)
```

Key differences:

- NumPy: soft clip (lossy but controlled)
- OpenCV: saturating integer (no wrapping, but less flexible)
- Division: clipping **hides** meaningful signal; prefer `normalise()`

Change detection: $|A - B|$ is near-zero everywhere the scene is unchanged.

4 Part 1C Logical Operations

Best on binary images (0 / 255)

```
# NumPy
np.where((a==255) & (b==255), 255, 0) # AND
np.where((a==255) | (b==255), 255, 0) # OR
np.where((a==255) ^ (b==255), 255, 0) # XOR

# OpenCV
cv2.bitwise_and(a, b)
cv2.bitwise_or(a, b)
cv2.bitwise_xor(a, b)
```

Goal	Operator
Consensus (fewest FP)	AND
Union (fewest FN)	OR
Disagreement / edges	XOR
Extract ROI	AND(mask, original)

5 Part 1D Otsu Thresholding

Goal: Automatically find t^* that maximises between-class variance:

$$\sigma_b^2(t) = \omega_0 \omega_1 (\mu_0 - \mu_1)^2$$

```
# NumPy (from scratch)
hist = np.bincount(img.ravel(), minlength=256).astype(np.float64)
p = hist / hist.sum()
omega = np.cumsum(p)
mu = np.cumsum(p * np.arange(256))
mu_T = mu[-1]
w0, w1 = omega, 1-omega
mu0 = mu/(w0+1e-12); mu1=(mu_T-mu)/(w1+1e-12)
t_star = int(np.argmax(w0*w1*(mu0-mu1)**2))

# OpenCV
t, bimg = cv2.threshold(img, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Works best: near-bimodal histogram, uniform lighting, moderate noise.

Fails: non-uniform illumination, many histogram modes, tiny foreground.

6 Integrated Pipeline (Part 1)

```
# Exercise 1: Brighten then Otsu
step1 = gamma_correction_np(img, 0.6)
t, mask = cv2.threshold(step1, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

```
# Exercise 2: ROI Extraction
roi = cv2.bitwise_and(img_gray, mask)
```

```
# Exercise 3: Change Detection
diff = cv2.absdiff(imgA, imgB)
t, change_mask = cv2.threshold(diff, 0, 255,
                               cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

```
# Exercise 4: Mask Fusion
or_mask = cv2.bitwise_or(mask1, mask2) # candidates
and_mask = cv2.bitwise_and(mask1, mask2) # consensus
xor_mask = cv2.bitwise_xor(mask1, mask2) # disagreement
```

7 Part 2 2D Convolution Theory

$$y[i, j] = \sum_u \sum_v K[u, v] \cdot x[i + u - \Delta_h, j + v - \Delta_w]$$

$$\Delta_h = \lfloor k_H/2 \rfloor, \quad \Delta_w = \lfloor k_W/2 \rfloor$$

Convolution vs Correlation: True conv flips kernel 180°. `cv2.filter2D` is cross-correlation (no flip). For **symmetric kernels**: identical. Flip manually for asymmetric (e.g. Sobel).

7.1 Padding Modes

Mode	Output size	Border
VALID	$(H - k + 1) \times (W - k + 1)$	Discarded
SAME zero	$H \times W$	Zeros outside
SAME reflect	$H \times W$	Mirrored
SAME replicate	$H \times W$	Repeat edge pixel

After a single 3×3 VALID conv: size shrinks by 2.

After n such layers: shrinks by $2n$. $\Rightarrow$ CNNs use SAME.

8 Part 2 NumPy Implementation

```
# VALID (no padding)
def conv2d_valid_np(img_u8, kernel):
    img_f = img_u8.astype(np.float32)
    k = kernel[:, :, :-1] # flip for true conv
    kH, kW = k.shape
    oH, oW = img_f.shape[0]-kH+1, img_f.shape[1]-kW+1
    out = np.zeros((oH, oW), np.float64)
```

```

for i in range(oH):
    for j in range(oW):
        out[i,j] = np.sum(img_f[i:i+kH,j:j+kW]*k)
    return normalise(out)

```

```

# SAME (with padding)
def conv2d_same_np(img_u8, kernel, pad='reflect'):
    img_f = img_u8.astype(np.float32)
    k = kernel[::-1,::-1]
    kH,kW = k.shape
    pH,pW = kH//2, kW//2
    padded = np.pad(img_f,((pH,pH),(pW,pW)), mode=pad)
    H,W = img_f.shape
    out = np.zeros((H,W), np.float64)
    for i in range(H):
        for j in range(W):
            out[i,j]=np.sum(padded[i:i+kH,j:j+kW]*k)
    return normalise(out)

```

9 Part 2 OpenCV Implementation

```

# SAME mode, normalised display
def conv2d_cv2(img_u8, kernel,
               border=cv2.BORDER_REFLECT):
    out = cv2.filter2D(img_u8.astype(np.float32),
                       cv2.CV_32F,
                       kernel.astype(np.float32),
                       borderType=border)
    return normalise(out.astype(np.float64))

```

```

# borderType options:
# cv2.BORDER_CONSTANT (zero pad)
# cv2.BORDER_REFLECT (mirror)
# cv2.BORDER_REPLICATE (edge repeat)

```

10 Part 2 Kernel Reference

```

K_BOX3 = np.ones((3,3))/9 # avg blur 3x3
K_GAUSS3 = np.array([[1,2,1],[2,4,2],
                     [1,2,1]])/16.0 # Gaussian
K_SHARPEN = np.array([[0,-1,0],
                     [-1,5,-1],[0,-1,0]]) # sharpen
K_SOBEL_X = np.array([[-1,0,1],
                     [-2,0,2],[-1,0,1]]) # horiz grad
K_SOBEL_Y = np.array([[-1,-2,-1],
                     [0,0,0],[1,2,1]]) # vert grad
K_LAP = np.array([[0,1,0],[1,-4,1],

```

[0,1,0]]) # Laplacian

Kernel type	Frequency response
Average / Gaussian	Low-pass (blur)
Sharpen	High-pass (boost edges)
Sobel X/Y	Band-pass directional
Laplacian	High-pass isotropic
LoG	Gaussian + Laplacian

10.1 Sobel Magnitude

```

Gx = cv2.filter2D(img.astype(np.float32),
                  cv2.CV_32F, K_SOBEL_X)
Gy = cv2.filter2D(img.astype(np.float32),
                  cv2.CV_32F, K_SOBEL_Y)
mag = np.sqrt(Gx**2 + Gy**2)
mag_u8 = normalise(mag)

```

11 Part 2 Full Pipeline

```

# Blur -> Gamma -> Otsu -> ROI
step1 = conv2d_same_np(img, K_GAUSS5, 'reflect')
step2 = gamma_correction_np(step1, 0.6)
t, mask = cv2.threshold(step2, 0, 255,
                        cv2.THRESH_BINARY+cv2.THRESH_OTSU)
roi = cv2.bitwise_and(img, mask)

```

Why blur first? Removes texture noise $\Rightarrow$ histogram has fewer spurious peaks $\Rightarrow$ Otsu finds a cleaner boundary.

12 Key Theory to Remember

Gamma correction: $s = r^\gamma$. Fixed points at $r = 0$ and $r = 1$. $\gamma < 1$ raises midtones; $\gamma > 1$ lowers them.

Otsu: Maximises $\sigma_b^2 = \omega_0\omega_1(\mu_0 - \mu_1)^2$. Assumes bimodal histogram. Equivalent to minimising within-class variance.

Convolution = weighted sliding window. Kernel determines the operation type:

- Sum of kernel = 1 $\Rightarrow$ no brightness change (blur)
- Sum of kernel = 0 $\Rightarrow$ detects change (edges)
- Sum of kernel $> 1 \Rightarrow$ brightens output

VALID vs SAME: VALID output size = $(H - k + 1) \times (W - k + 1)$. SAME output size = $H \times W$ (padding required).

Box vs Gaussian blur: Box blur: equal weights ringing artefacts. Gaussian: smooth bell-curve weights fewer artefacts.

Laplacian is noisy. Always apply Gaussian first (= LoG) for reliable edge detection.

Clip vs Normalise for display:

- Clip: correct for filters where output [0,255] (blur, sharpen)
- Normalise: needed for edge filters (contain negatives / values > 255)

np.convolve / scipy.signal vs cv2.filter2D: NumPy/SciPy flipkernels. OpenCV does not. For symmetric kernels: same result.

13 Common Mistakes to Avoid

- Doing `img1 + img2` on `uint8` directly wraps around (e.g. $200 + 100 = 44$). Always convert to float first.
- Using `VALID` mode then expecting the same spatial size as the input.
- Applying Laplacian directly to a noisy image (amplifies noise). Use LoG instead.
- Using `cv2.filter2D` for asymmetric kernels without flipping (cross-correlation convolution for asymmetric kernels).
- Normalising sharpened output: use clip, not normalise, or sharpening appears to change brightness.